

Introduction to Database Systems

Spring 2025

Lecture 6

Using SQL in Applications

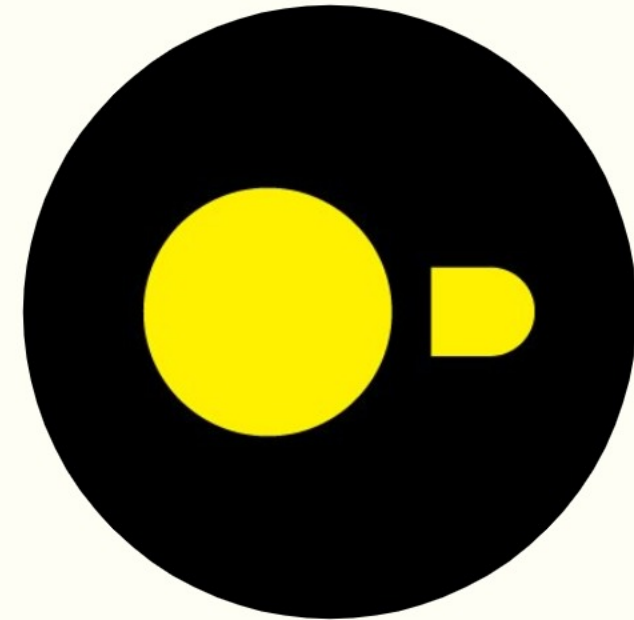
Omar Shahbaz Khan

General Info

- Homework 2 due today (March 5th, 23:59)
 - Make sure to **SUBMIT** and not just saved as a draft
 - Unsubmitted assignments will not be graded!

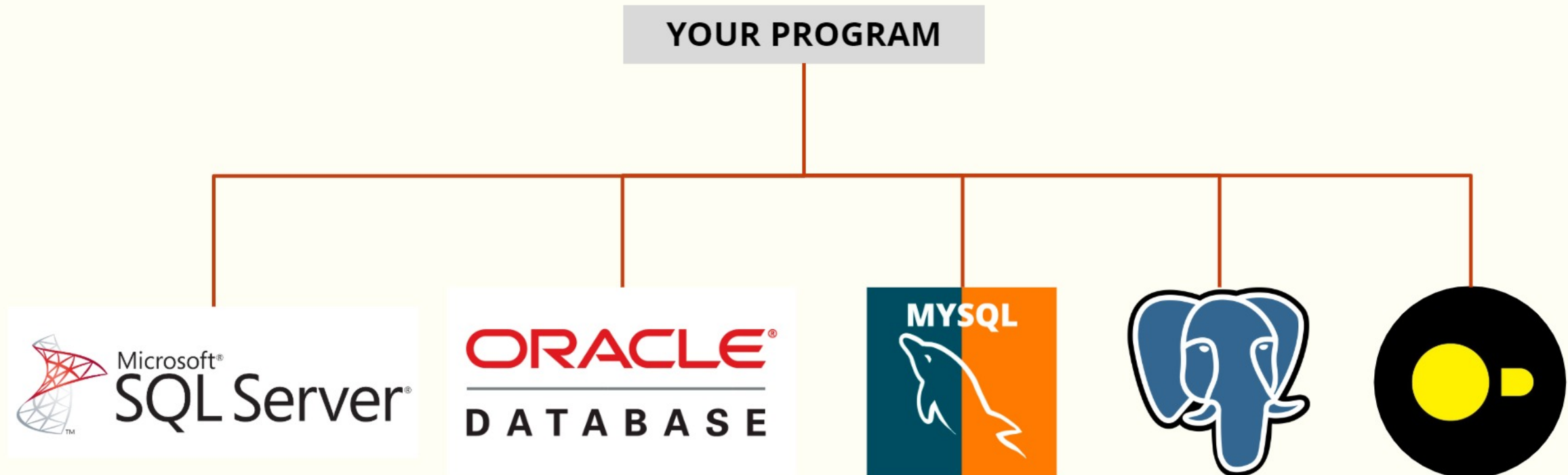
TODO

- SQL in Python
 - ODBC API
 - Basic Usage
- SQL Injections
 - Prepared Statements
- Fullstack Example Application



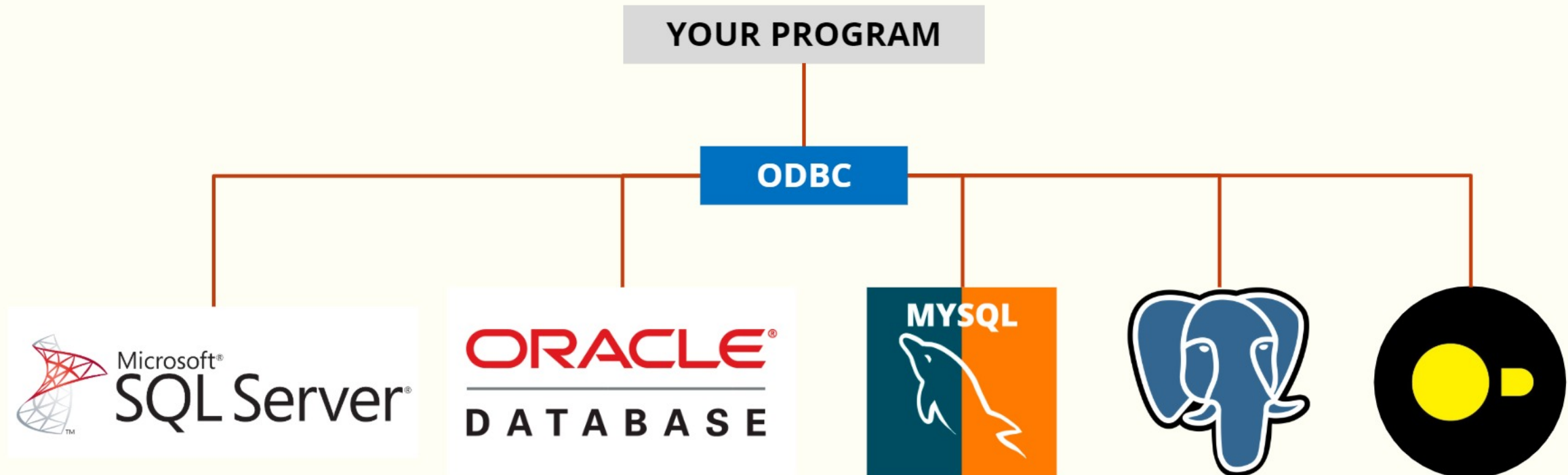
Problem: Vendor Lock-in

- They're all compatible with standard SQL...
- ...and mutually incompatible due to vendor-specific functionality!



Open Database Connectivity (ODBC)

- In-between translation layer
- Vendors not so happy as lock-in is profitable!



Why connect to DB with another language?

- Python / PERL / PHP are used to define the middle layer in a web stack (recall LAMP)
- Desktop Apps developed in Java or C# similarly access databases using JDBC / ADO.NET / Entity Framework
- Automation for data loading / transformation / storing data from periodical jobs
 - [Tower](#) is a recent startup for exactly this type of ETL/ELT workload with Python

Python and DuckDB

- pyodbc is a package that implements ODBC and can connect to any databases
- The duckdb package is a database adapter specifically for DuckDB
 - To install: `python -m pip install duckdb` or `pip install duckdb`
 - `import duckdb`
 - Similar specialized package exists for other vendors as well
 - Snowflake: `snowflake-connector-python`
 - PostgreSQL: `psycopg`
- A simple database API
 - `connect()`: Establish connection to specified database
 - `cursor()`: Object to manage context of an SQL operation
 - `execute()`: Execute SQL queries
 - `fetchone()`, `fetchmany()`, `fetchall()`: Get results from queries
 - Much more, but these are the most used functions
 - For the specialized packages there could be some differences so check [documentation](#)

Connecting to a Database

```
import duckdb

conn_string = "<local_db_file>" # or ":memory:" | "url to file"

# Connection option one
conn = duckdb.connect(conn_string, read_only=False)

# Execute Queries using Cursors
conn.close()

# Connection option two
with duckdb.connect(conn_string, read_only=False) as conn:
    # Execute Queries using Cursors
# Closes connection
```

Tips

```
try:
    # Connection option one
    conn = duckdb.connect(conn_string, read_only=False)
    # Execute Queries using Cursors
except Exception as error:
    print(error)
finally:
    conn.close()
```



This block is always run

```
try:
    # Connection option two
    with duckdb.connect(conn_string, read_only=False) as conn:
        # Execute Queries using Cursors
        # Closes connection
except Exception as error:
    print(error)
```

Querying the DB

```
# Get all athletes
```

```
athletes: list = conn.execute("SELECT * FROM Athletes").fetchall()
```

Executes the query and gets all athletes

```
# Get five athletes
```

```
athletes: list = conn.execute("SELECT * FROM Athletes").fetchmany(5)
```

Get set number of rows or a single row

```
# Get one athlete
```

```
athletes: list = conn.execute("SELECT * FROM Athletes").fetchone()
```

```
# Get different number of rows from the same query
```

```
conn.execute("SELECT * FROM Athletes") # Execute query once
```

```
first_five = conn.fetchmany(5) # Fetch first 5 rows
```

```
sixth = conn.fetchone() # Fetch sixth row
```

```
the_rest = conn.fetchall() # Fetch the remaining rows
```

Cursors

- A database object to run and manage query results
- Useful for parallel access to the database
- You can get a cursor in the following manner
 - `cur = conn.cursor()`
 - `cur = conn.cursor().execute(...)`
 - `cur.fetchone()`, `cur.fetchmany(...)`, `cur.fetchall()`
- Remember to close cursors as they take up resources or may hold locks on tables if their query has not fetched all rows
 - `cur.close()`

Cursors

```
# Get query cursor
athletes_cursor = conn.cursor().execute("SELECT * FROM Athletes")

# Get five athletes
athletes: list = athletes_cursor.fetchmany(5)

sports_cursor = conn.cursor().execute("SELECT * FROM Sports")

# Get sixth athlete
athlete_six = athletes_cursor.fetchone()

sports: list = sports_cursor.fetchall()

# Get the next 10 athletes
athlete_next_ten = athletes_cursor.fetchmany(10)

athletes_cursor.close()
sports_cursor.close()
```

Advantage: Multiple
cursors active at the same
time, which can fetch or
execute queries

Remember to close the cursors

Creating a Table

```
conn.execute("DROP TABLE IF EXISTS Users")
conn.execute("DROP SEQUENCE IF EXISTS test_id_seq")
conn.execute("CREATE SEQUENCE test_id_seq START 1")
conn.execute(
    """
    CREATE TABLE Users (
        id INT PRIMARY KEY DEFAULT nextval('test_id_seq'),
        name VARCHAR(100) NOT NULL,
        password VARCHAR(250) NOT NULL,
        role_name VARCHAR(50)
    )
    """
)
```

INSERT

```
with conn:
    conn.execute(
        """
        INSERT INTO Users (name, password, role_name)
        VALUES ('Omar', '****', 'viewer')
        """
    )
```

Using a with context block on a connection instance,
closes the connection when leaving the block

```
other = conn.execute("SELECT 42 as universe").fetchone()
```


Let's not use plain password

```
conn.execute("DROP TABLE IF EXISTS Users")
conn.execute("DROP SEQUENCE IF EXISTS test_id_seq")
conn.execute("CREATE SEQUENCE test_id_seq START 1")
conn.execute(
    """
    CREATE TABLE Users (
        id INT PRIMARY KEY DEFAULT nextval('test_id_seq'),
        name VARCHAR(100) NOT NULL,
        password_hashed VARCHAR NOT NULL,
        role_name VARCHAR(50)
    )
    """
)
```

Creating hashed password with bcrypt

```
import bcrypt
password = '****'
salt = bcrypt.gensalt()
hashed_pw = bcrypt.hashpw(password.encode('utf-8'), salt)
hashed_pw_str = hashed_pw.decode('utf-8')

def hash_password(password: str) -> str:
    salt = bcrypt.gensalt()
    hashed_pw = bcrypt.hashpw(password.encode('utf-8'), salt)
    return hashed_pw.decode('utf-8')
```

INSERT with variables

```
name = 'Omar'
password_hash = hash_password('****')
role = "'viewer'"

conn.execute(
    """
    INSERT INTO Users (name, password_hashed, role_name)
    VALUES ('%s', '%s', %s)
    """ % (name, password_hash, role)
)
```

Prone to SQL Injections!

Login

Username
Viewer

Password
.....

LOGIN



Login

Username

Password

LOGIN



```
def check_user_credentials (
    username:str,
    password:str
) -> str:
    try:
        user = conn.execute(
            """
            SELECT * FROM Users WHERE username='%s'
            """ % username
        ).fetchdf().to_dict(orient="records")[0]

        if bcrypt.checkpw(
            password.encode('utf-8'),
            user['password_hashed'].encode('utf-8')
        ):
            return user
    except Exception as e:
        print(e)
        return None
```

Dangers of SQL Injection

- SQL Injections can lead to data exfiltration
- Add corrupted data
- Delete data and drop database elements (schemas/tables/roles/etc.)
- If the active database user has high privileges they can even drop databases

SQL Injection Example (% replacement)

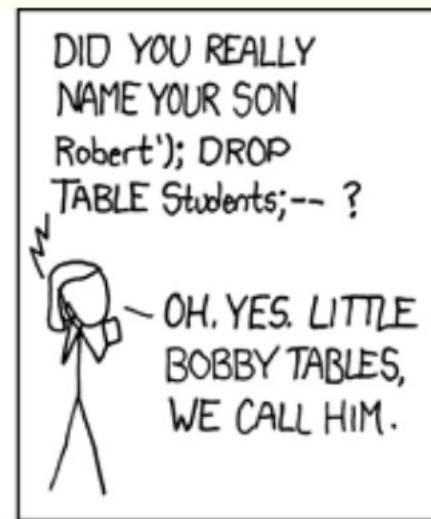
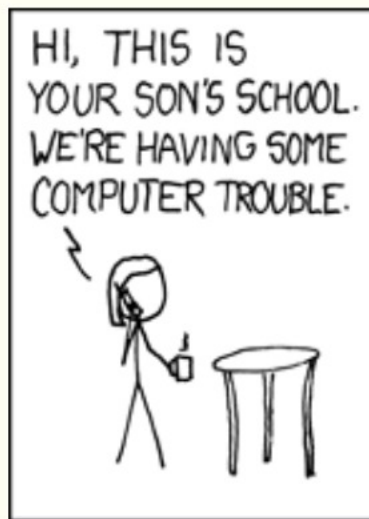
```
name = "Omar', ' ', ' '); CREATE TABLE attack (text VARCHAR(50)); --"
password_hash = hash_password('****')
role = "'viewer'"

conn.execute(
    """
    INSERT INTO Users (name, password_hashed, role_name)
    VALUES ('%s', '%s', %s)
    """ % (name, password_hash, role)
)
```


SQL Injection Example (f"{variable}")

```
name = "'Omar'"
password_hash = hash_password('****')
role = "'viewer'); INSERT INTO attack VALUES ('haha'); --"

conn.execute(
    f"""
    INSERT INTO Users (name, password_hashed, role_name)
    VALUES ({name}, '{password_hash}', {role})
    """
)
```



SQL Injections, still attempted in the future?



**Star Trek Discovery
Season 2 Episode 8**

Year 2255+

Prepared Statements

- Separate the query string and variables
- Prepared Statements
 - Pre-compiled SQL statement at DBMS
 - Client supplied data is treated as content of a parameter and not an SQL statement

```
PREPARE my_query AS  
SELECT *  
FROM Users  
WHERE name = "$1";
```

```
EXECUTE my_query('Probe used SQL Injection... it's not very effective');
```

Prepared Statements in Python

```
name = 'Omar'
password_hash = hash_password('****')
role = "'viewer'); DROP TABLE Users; --"

conn.execute(
    """
    INSERT INTO Users (name, password_hashed, role_name)
    VALUES (?, ?, ?)
    """
    ,
    [name, password_hash, role]
)
```

Not safe against all cases!

- Some elements can't be prepared
- SQL Statements that consider the column names
 - INSERT
 - ORDER BY
- Check the input before using the variable

```
sort_key = "name"

if sort_key in columns:
    cur = conn.execute(
        f"""
        SELECT *
        FROM Athletes
        ORDER BY {sort_key}
        """
    )

    athletes = cur.fetchall()
```


From DB to Web

- DuckDB Database
 - duck_sports_ddl.sql
 - duck_users_ddl.sql
- Python Server
 - Serves an API using FastAPI and Uvicorn
 - Calls functions from the DatabaseAPI
 - DatabaseAPI is a custom python class that performs queries on the database
- Website
 - Vuejs with Vuetify (typescript)
 - Calls the server with axios

Database Schema

- Sports (ID, name, record)
- Athletes (ID, gender, name, height)
- Gender (gender, description)
- Competitions (ID, place, held)
- Results (athleteID, competitionID, sportID, result)
- Roles (role_name)
- Users (ID, name, password_hash, role_name)

Roles and Users

- Roles
 - viewer: Only allowed to view data
 - editor: Allowed to view, add, and update data rows
 - admin: Allowed to view, add, update, and delete data and tables
- There are 3 users, whose name and password are the roles with capital first letter
 - See `duck_users_ddl.sql`
- DuckDB has no way of enforcing such behavior at the database level
 - We have to enforce it in the application code
 - Snowflake, PostgreSQL, MySQL, etc. can do this with Role Based Access Control (RBAC)
 - RBAC will be the focus of next week's lecture

Sports

View all the sports

Athletes

These athletes have taken part in the competitions

Competitions

Take a look at all these competitions that have taken place

Results

Glance at the results from specific places and sports

Add a new Athlete

Uses a function from SQL (PostgreSQL) or Python (DuckDB)

Add a new Competition

Uses INSERT INTO from pycogp or duckdb

Delete a Sport

Better reset the DB at some point

How to run it

- Download the fullapp.zip file from LearnIT
 - Extract it into a directory of your choice
- Database (/sql)
 - Navigate to the *sql* directory from the terminal
 - Run: `duckdb < duck_sports_ddl.sql sportsweb.db`
 - Run: `duckdb < duck_users.sql sportsweb.db`
- Python Server (/duck-api)
 - Navigate to the *duck-api* directory from the terminal
 - *Optional*: Create a virtual environment for this project (poetry, conda, venv)
 - Run: `python -m pip install -r requirements.txt`
 - Run: `python Server.py <duckdb_file>`
- Web page (/dist)
 - Navigate to the *dist* directory from another terminal (one for the server, one for the web app)
 - Run: `python -m http.server 8000`
 - You can now access the web page at `http://localhost:8000`

Using fetchdf()

- With DuckDB you can also fetch rows as a Pandas Dataframe
- Handy for analytical processing
- Modifying output
- Example:
 - I want to round my floats to 2 decimals and return the rows as a list of dictionaries

```
rows = self.connection.execute("SELECT id, name, record FROM Sports").fetchdf()  
rows['record'] = rows['record'].apply(lambda x: round(x,2))  
rows = rows.to_dict(orient="records")  
return rows
```


Takeaways!

- ODBC (Python: pyodbc, Java JDBC)
 - API for database operations
 - Specific database adapter packages also exists
 - duckdb (DuckDB)
 - psycopg (PostgreSQL)
 - snowflake-connector-python (Snowflake)
 - Connect to the database and execute queries directly or through cursors
- SQL Injections
 - Attacks from the variables used in SQL queries
 - Prepared Statements, separate query string and variables